

Enhancing neural network explainability via event causal graphs for bottleneck prediction in production systems

Weihong Cen^a , Lie Yang^b , Chupeng Su^a, Gang Chen^{a,*} , Longhan Xie^{a,*}

^a Shien-Ming Wu School of Intelligent Engineering, South China University of Technology, Guangzhou, 511464, Guangdong, China

^b School of Mechanical and Aerospace Engineering, Nanyang Technological University, 637460, Singapore

HIGHLIGHTS

- An event causal graph neural network is proposed based on event-feature causal analysis to represent system states.
- Transformer-based multi-head attention and graph attention mechanisms are integrated to capture short- and long-term event dependencies.
- Model interpretability and predictive performance are significantly enhanced through the proposed approach.

ARTICLE INFO

Keywords:

Bottleneck prediction
Production system
Bottleneck spatio-temporal neural network
with event causal graph
Graph neural network
Transformer

ABSTRACT

Bottleneck prediction is essential for improving throughput in production systems and is a key research focus. However, the dynamic nature of bottleneck locations, driven by complex and evolving production environments, presents significant challenges in this field. Existing methods often struggle to capture the intricate relationships among influencing factors, leading to limited interpretability and predictive accuracy. To address these issues, we propose an Event causal graph-based spatial-temporal neural network (ECGBSTNN) for bottleneck prediction. This approach represents machine operations and buffer states as event nodes, integrating expert knowledge to construct causal links, forming a dynamic, event-level causal graph. In addition, a weight matrix quantifies uncertain causal relationships, facilitating adaptive and interpretable causal weighting for bottleneck prediction. By integrating a Graph Neural Network (GNN) with a Transformer-based architecture, ECGBSTNN captures both causal and temporal dependencies among events to predict future blockage and starvation times which are leveraged to identify potential bottlenecks. Experiments on a production system demonstrate that ECGBSTNN outperforms state-of-the-art methods, for blockage prediction, while providing enhanced interpretability and practical applicability.

1. Introduction

In the face of intensifying market competition, enhancing throughput is critical for manufacturing industries to bolster operational efficiency and competitiveness [1,2]. However, throughput improvement is hindered by bottlenecks that constrain production efficiency [3]. In discrete manufacturing systems, a bottleneck is defined as the station that limits overall throughput. According to the theory of constraints, mitigating bottlenecks can significantly enhance system performance and increase throughput [4]. Consequently, researchers have focused on both historical and real-time bottleneck detection to optimize manufacturing operations [5–7]. Although detection identifies current issues, it cannot forecast future conditions, making bottleneck prediction increasingly

essential. The approach of prediction has gained prominence, enabling proactive operational adjustments and precise maintenance scheduling to improve production efficiency [8,9].

Bottleneck detection in manufacturing primarily relies on simulation-based and statistical analysis methods. Simulation-based approaches [10,11] model production processes to identify potential bottlenecks by simulating operational dynamics. The complexity and variability of real production environments often lead to significant deviations from actual conditions, which limits the accuracy of these methods in real production environments. In contrast, analytical methods [12–14] utilize real-time manufacturing data for statistical analysis to detect bottlenecks and track their historical shifts. While effective for identifying past and present bottlenecks, these methods

* Corresponding authors.

Email addresses: gangchen@scut.edu.cn (G. Chen), melhxie@scut.edu.cn (L. Xie).

<https://doi.org/10.1016/j.asoc.2026.115795>

Received 21 October 2025; Received in revised form 8 May 2026; Accepted 18 June 2026

Available online 20 June 2026

1568-4946/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

struggle to predict future bottlenecks in dynamic environments, hindering proactive scheduling and maintenance planning. Therefore, developing predictive methods to forecast bottleneck machines and enable proactive task adjustments is critical for enhancing production throughput and operational efficiency.

However, the dynamic complexity of manufacturing systems, driven by factors such as machine downtime, production schedules, and operator skills, generates high-dimensional data, which poses significant challenges for bottleneck prediction [15]. Research has progressed from statistical models [16] to advanced machine learning techniques, such as LSTM [17,18]. However, three critical research gaps remain. Processing high-dimensional data with both accuracy and scalability remains underexplored, as current methods struggle to balance computational efficiency with predictive precision in real-time applications. While short-term temporal dynamics are effectively modeled, capturing long-term temporal dependencies, which are crucial for predicting delayed bottleneck shifts [19], is inadequately addressed. This limitation restricts foresight in dynamic systems. Spatial-temporal neural networks capture station-level correlations [15,20,21], but feature-level relationships, such as interactions between specific variables like downtime and queue length across stations [22], are rarely studied. Feature-level relationships refer to interactions between individual variables (e.g., upstream downtime affecting downstream queue length), in contrast to station-level correlations that focus on broader inter-station dynamics. Addressing these challenges, including scalable high-dimensional data processing, robust long-term temporal modeling, and comprehensive feature-level relationship mapping, is essential for enhancing the accuracy and applicability of bottleneck predictions in manufacturing systems.

To address these gaps, developing a comprehensive understanding of manufacturing processes is essential, systematically modeling causal, temporal, and dependency relationships among features to capture time-evolving dynamics [23]. In this context, events serve as effective abstractions for system changes, enabling interpretable feature-level interactions and temporally informed bottleneck prediction. Events, such as machine failures or schedule adjustments, depend on variables like downtime duration or working time, making them a robust choice for modeling feature-level relationships and their temporal dependencies in dynamic manufacturing systems.

Events abstract feature interactions and temporal dynamics in manufacturing systems. Analyzing their behavior reveals causal and sequential relationships, supporting accurate bottleneck prediction and system optimization. In manufacturing systems, events such as process initiation, completion, material flow, and machine state changes are recorded in log data, providing fine-grained insights into system dynamics and facilitating bottleneck prediction [24]. These events exhibit inherent causal and temporal sequential properties, reflecting the logical structure and time-evolving dynamics of production systems. For instance, a delay in material delivery from an upstream station can trigger starvation at a downstream station, illustrating causal dependencies that influence bottleneck formation. In complex discrete manufacturing systems, capturing these spatial and contextual relationships among events is critical for improving the accuracy and robustness of bottleneck predictions. However, the high dynamism and uncertainty of events, driven by factors such as variable production schedules, machine downtime, and operator performance [15], pose significant challenges for predictive models. Moreover, sequential causal chains formed by event interactions often exhibit delayed effects, where historical events continuously shape subsequent system states, complicating long-term dependency modeling. Consequently, advanced methods that integrate spatial-temporal causal relationship modeling, such as graph neural networks (GNNs) for spatial interactions [20] and modified Transformers for temporal dynamics [25], are essential to accurately predict and manage the evolving dynamics of production systems, enabling proactive maintenance and operational optimization.

Motivated by the above identified challenges and analysis, we propose a Spatial-Temporal Neural Network with Event Causal Graphs (ECGBSTNN) for bottleneck prediction in complex systems. To effectively capture causal and temporal dependencies among event features, we integrate Graph Neural Networks (GNNs) to model causal relationships and Transformer-based methods to capture temporal trends. Events are represented as nodes in a dynamic, weighted, directed graph, with edges constructed based on dynamic causal interactions. These interactions account for uncertainties in event responses, and following [26], a causal weight matrix is utilized to encode dependencies and their associated uncertainties. The event graph is processed using an attention-based GNN, which is combined with a Transformer-based temporal model to jointly capture dynamic causal relationships and long-term temporal patterns. This approach enables accurate prediction of station blockage and starvation, which are critical indicators for bottleneck identification, as established in [16]. Finally, we employ the turning point method [27], a robust data-driven technique, to identify future bottlenecks based on the predicted blockage and starvation patterns.

In summary, the main contributions of our work are as follows:

1. To the best of our knowledge, the ECGBSTNN framework is the first to leverage causal event relationships and their temporal dynamics for interpretable bottleneck prediction in complex manufacturing systems.
2. Events are used to represent feature-level interactions and are modeled as graph nodes, with edges capturing their causal and sequential dependencies. This reveals system dynamics and improves interpretability and prediction accuracy.
3. The proposed method uses Graph Neural Networks to model dynamic causal event relationships and a Transformer with a causal mask to capture long-term temporal dynamics in a production system.

The paper is structured as follows: [Section 2](#) reviews bottleneck prediction and graph neural network advancements. [Section 3](#) describes the proposed spatio-temporal neural network. [Section 4](#) presents a production line case study and benchmark comparisons. [Section 5](#) concludes the paper.

2. Related work

The section provides a structured overview of the research background and recent developments. [Section 2.1](#) reviews the evolution from bottleneck detection to prediction, highlighting the limitations of existing temporal and spatiotemporal methods, and motivating event-level causal modeling with GNNs and Transformers to learn event causality and long-term dependencies. [Section 2.2](#) focuses on event-driven approaches, discussing causal graph learning and long-term dependency modeling, and emphasizing the effectiveness of adaptive adjacency matrices and attention mechanisms.

2.1. Bottleneck detection and prediction

In manufacturing systems, bottlenecks refer to machines that limit overall throughput [28]. Bottlenecks are affected by multiple factors, and various methods have been applied to mitigate them and enhance throughput. Analytical approaches [13,29] rely on theoretical equations, while simulation-based methods [10,11] use discrete event simulations to identify bottlenecks. However, both face challenges in dynamic environments due to their limitations in adapting to changing conditions. Data-driven approaches [12,14] are better suited for dynamic production environments, leveraging real-time historical production data to analyze bottlenecks. Over the past decade, data-driven methods have demonstrated significant efficacy in both academic and industrial settings by leveraging real-time production data to detect

and mitigate bottlenecks, outperforming conventional analytical and simulation-based approaches. However, these methods are largely retrospective, providing limited predictive capability for system states, thus constraining operators' ability to plan and optimize throughput. As a result, research has increasingly shifted toward developing bottleneck prediction techniques.

Many efforts aim to provide predictive insights for proactive system optimization. Most existing approaches directly input raw data into predictive models, which autonomously learn data relationships and input–output mappings. Traditional methods model these relationships by extracting rules from the data. The ARMA-based methods [30,31] leverage linear combinations to model time series, suitable for simple, stationary data. In contrast, ANFIS [17] utilizes fuzzy rules and neural networks to capture nonlinear input–output mappings, effectively handling complex production data. However, these methods face challenges with nonlinear dynamics and demand intensive computational resources for high-dimensional features. To address the limitations of traditional methods, researchers have increasingly employed machine learning techniques to model data relationships and capture temporal dependencies in time series, based on their robust feature extraction and generalization abilities. Lai et al. [32] employ a Long Short-Term Memory (LSTM) network to process high-dimensional data for predicting future bottlenecks, integrating historical data with future production planning inputs. Fang et al. [19] proposed a parallel gated recurrent units (P-GRUs) network to extract the temporal correlations from main inputs and auxiliary inputs for bottleneck prediction. Liu et al. [18] verified the roles of different features in the prediction using a Long Short-Term Memory network with a dual attention mechanism (LSTM-DAM). Although the above have employed machine learning to predict time-evolving production bottlenecks by capturing temporal correlations in high-dimensional historical data, they often overlook the feature relationships associated with production line layout and material flow.

Researchers have found that production data not only exhibit dynamic temporal dependencies but also, with deeper investigation, reveal inherent spatial features determined by manufacturing processes. Consequently, related research methods have gradually evolved from temporal prediction approaches to spatial-temporal models that integrate both temporal dependencies and spatial relationships. They predict the bottleneck indicators in the future period and leverage them to identify the future bottlenecks in a manufacturing system. Ma et al. [22] highlighted that production data exhibit temporal and spatial relationships across stations, with shifting bottlenecks influenced by these temporal-spatial features. Consequently, they proposed the TUSB-LSTM (integrated LSTM with Bi-LSTM) method to capture temporal relationships within individual station features and spatial correlations among stations. However, the spatial correlations captured by these models may lack accuracy. Lai et al. [15] mapped an assembly line layout into a graph based on the material flow and proposed a BSTAN (integrating the graph attention neural network, GAT and gated recurrent unit, GRU) to extract the spatial dependencies among stations using GAT and capture the time-evolving dynamics of features. Besides, Yin et al. [20] noted that a change in one task can trigger changes in others and proposed a spatio-temporal graph convolutional network (BTGCN) to model the cascading effects and temporal dependencies in the data. Moreover, Wu et al. [21] argued that the primary challenge in bottleneck prediction stems from effectively utilizing knowledge of sustainable production processes. They proposed the BPSTK method, which constructs a knowledge graph of production processes and resources, integrating GAT with GRU to predict bottlenecks. Moreover, our last work divided the features into temporal and spatial features and employed a parallel model (Dual-BDSTN [33]) to learn the temporal and spatial relationships. Although graph-based methods can extract spatial correlations by mapping sites or tasks to graph nodes, each node often encompasses multiple features, which may limit their ability to accurately capture the spatial relationships among these features. Therefore, it is essential to treat features as

graph nodes and model their transitive relationships to capture feature interactions effectively.

Above all, existing bottleneck prediction methods have evolved from purely temporal models to spatio-temporal frameworks that incorporate structural information. However, most approaches focus on station-level representations, where multiple features are aggregated within each node, limiting their ability to capture fine-grained feature interactions. In addition, these methods often lack explicit modeling of causal relationships and struggle to capture long-term temporal dependencies simultaneously. These limitations motivate the need for a more fine-grained, event-driven, and causality-aware modeling framework.

While modeling feature dependencies is essential, understanding the event-level interactions that drive these dynamics is equally critical for accurate bottleneck prediction. Previous research on the interactive dynamics between production events remains limited. The production rhythm of the system is captured through recorded events in log data, which provide finer granularity to accurately reflect the system's operational status [34]. In discrete manufacturing systems, events are independent and have sequential relations (causality effects) [23]. To accurately capture these effects, methodologies must integrate causal information inherent to the production system. The causal relationships among events are complex and non-trivial, influenced by diverse tasks and external disruptions that affect product flow. Moreover, variations in event response times induce temporal dependencies, reflecting the sequential dynamics inherent in the system. On the other hand, to effectively model long-term feature dependencies, the Transformer model [25] employs multi-head attention mechanisms, achieving superior performance in capturing temporal dynamics. Therefore, we can develop mappings of events into causal graphs by integrating expert system knowledge and employing graph neural networks to model dynamic event relationships, while designing modified Transformers to capture both long-term and short-term temporal dynamics for bottleneck prediction.

2.2. Capturing causal and temporal events using spatio-temporal neural networks

Event-driven approaches enhance the responsiveness and adaptability of equipment monitoring in manufacturing systems. Processing tasks are closely related to system states and can be captured through monitored events [35]. There exists a cause-effect associative evolutionary mechanism among them [36]. Modeling causal relationships between events as a knowledge graph aids in revealing underlying system mechanisms and has demonstrated effectiveness in applications like fault diagnosis, clinical medicine, and traffic networks. In manufacturing systems, event occurrences exhibit causal relationships similar to those observed in fault diagnosis and traffic networks. Therefore, events in manufacturing processes can also be mapped to nodes and their interactions represented as edges in a graph structure. Many approaches employ spatial-temporal neural networks to capture their patterns. They describe event relationships through dynamic semantic information [34]. While many leverage data-driven methods to assign the nodes and edges with weights, such as equal weights, rule-based weights [15,37] and learned weights [38,39].

In spatial-temporal networks, traditional GNNs with fixed equal edge weights inadequately capture dynamic event causality. Learnable edge weights are thus introduced to adaptively model time-varying causal dependencies, enhancing spatial-temporal representations. Graph Attention Network (GAT) introduces an attention mechanism that dynamically captures the importance of relationships between nodes. Such bidirectional weighting struggles to reflect the one-way nature of causal effects accurately. GCN weights are determined by calculating the spatial similarity of input features to integrate the graph and capture real-time changes in traffic states [39]. Causation entropy is applied to compute a weighted matrix for the GCN, enabling effective fusion of graph features

for prediction [40]. PSTCGCN [41] employs two learnable parameters to derive a weighted matrix for causal relationship fusion. DB-STGCN [42] integrates a dynamic Bayesian network with a spatio-temporal GNN to capture dynamic event causality for prediction. Due to its ability to model hyperparameter uncertainty, the approach can more accurately capture the directionality and inherent uncertainty of causal weights between events.

For capturing long-term dependencies in dynamic event sequences, traditional RNNs are limited by gradient explosion and vanishing, while the attention mechanism effectively models temporal relations influenced by historical events. TEDDCF [39] integrates multi-head attention and causal convolution to capture the long-term trends and short-term event dynamics in traffic flow prediction. TADGCN [43] combines attention mechanisms and CNNs to capture temporal correlations from various periodic traffic patterns. [44,45] employ the transformer to model temporal point processes and learn time-evolving dynamics. Therefore, the attention mechanism serves as an effective approach for modeling long-term event feature sequences.

Based on the above analysis, we construct a causal graph with dynamically learnable edge weights and integrate attention mechanisms to jointly model causal relationships and long-term temporal dependencies between events, enabling causal event-driven bottleneck prediction.

3. Methodology

In this section, we first provide the basic definitions, including event features, dynamic causal graph, and information sequence. Then, we introduce the approach for bottleneck identification and prediction, followed by detailed explanations of the proposed model.

3.1. Definitions

Event Features. The state of a station can be represented by four types of events: operating, blocked, starved, and down [27]. The buffer has three possibilities: full, partially full, and empty. Within a production period T , the time summation of each station can be expressed by Eq. (1), while the time of each buffer can be described with Eq. (2). Therefore, these events are selected as event features for models to predict bottlenecks.

$$T = TB_i + TD_i + TS_i + TW_i \quad (1)$$

where TB_i , TD_i , TS_i , and TW_i denote the blockage time, downtime, starvation time, and working time of each machine, respectively.

$$T = TF_i + TPF_i + TE_i \quad (2)$$

where TF_i , TPF_i and TE_i are the full time, partially full time, and empty time of each buffer, respectively.

Directed causal graph. An event causal graph can be represented as a directed graph $G = (\mathcal{V}, \mathcal{E}, \mathcal{A})$, where $\mathcal{V}$ denotes the set of $|\mathcal{V}| = N$ nodes, and each node is an event. $\mathcal{E}$ is the set of directed edges connecting nodes of causal events in a production system. $\mathcal{A} \in \mathbb{R}^{N \times N \times T_s}$ is the directed adjacency matrix of graph G , representing node relationships in historical time T_s steps. It can be given or learned from data.

Information sequence and prediction. To extract temporal features from information sequences, we adopt a sliding window to divide them into fixed-length subsequences, enabling the modeling of dynamic dependencies. Event time prediction employs the historical time T_s step event sequence $X_{in} = [X_{t-T_s+1}, \dots, X_{t-1}, X_t] \in \mathbb{R}^{N \times T_s}$ to predict the future event time of T_f steps $Y = [Y_{t-T_s+1}, \dots, Y_{t-1}, Y_t] \in \mathbb{R}^{N \times T_f}$. The process can be expressed as:

$$[Y_{t-T_s+1}, \dots, Y_{t-1}, Y_t] = F_{\Theta}([X_{t-T_s+1}, \dots, X_{t-1}, X_t], \mathcal{A}) \quad (3)$$

where F denotes a mapping function and Θ represents the learnable parameters.

3.2. Framework overview

The overall architecture of the proposed methodology is illustrated in Fig. 1. Firstly, a production line in the factory can be abstracted into a layout based on the material flow during the manufacturing process. Then, in production systems, stations typically exhibit four event states: operating, blocked, starved, or down. Buffers can be in one of three states: full, partially full, or empty. The changes and interactions between these states drive the dynamic transfer of bottlenecks and reveal the significant causal relationships and temporal dependencies between event states. Based on the causal relationships among events, we construct a directed event causal graph, shown in Fig. 2. Subsequently, the graph with a dynamic weight matrix is fed into the Event Causal Graph Attention mechanism to obtain higher

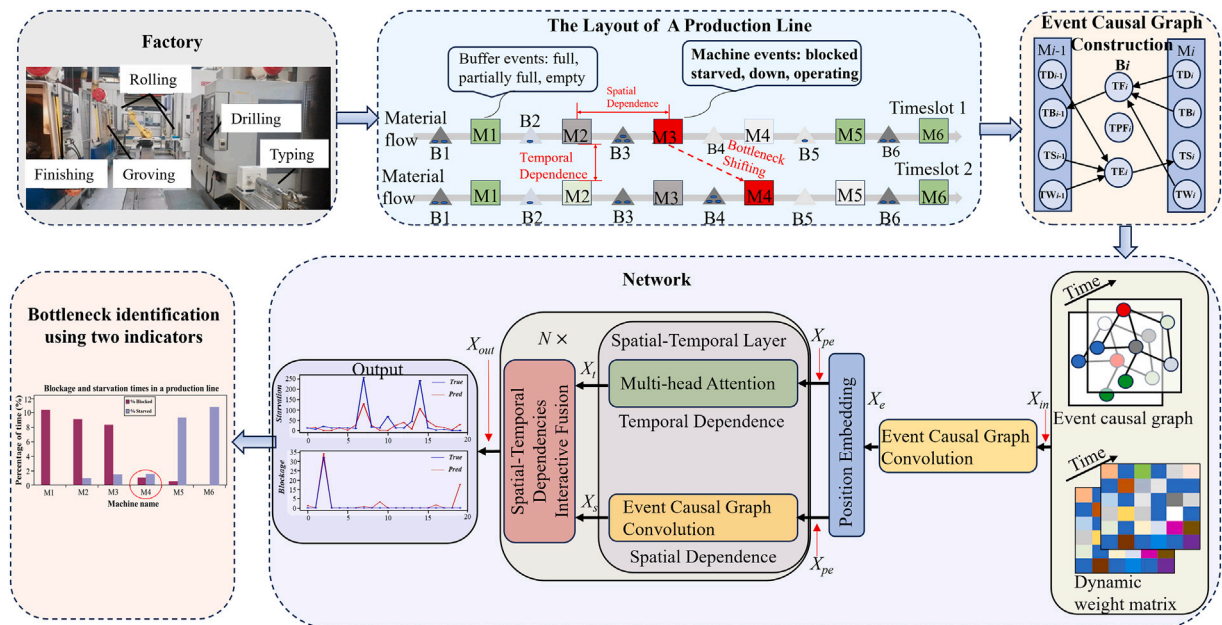


Fig. 1. The architecture of the proposed methodology.

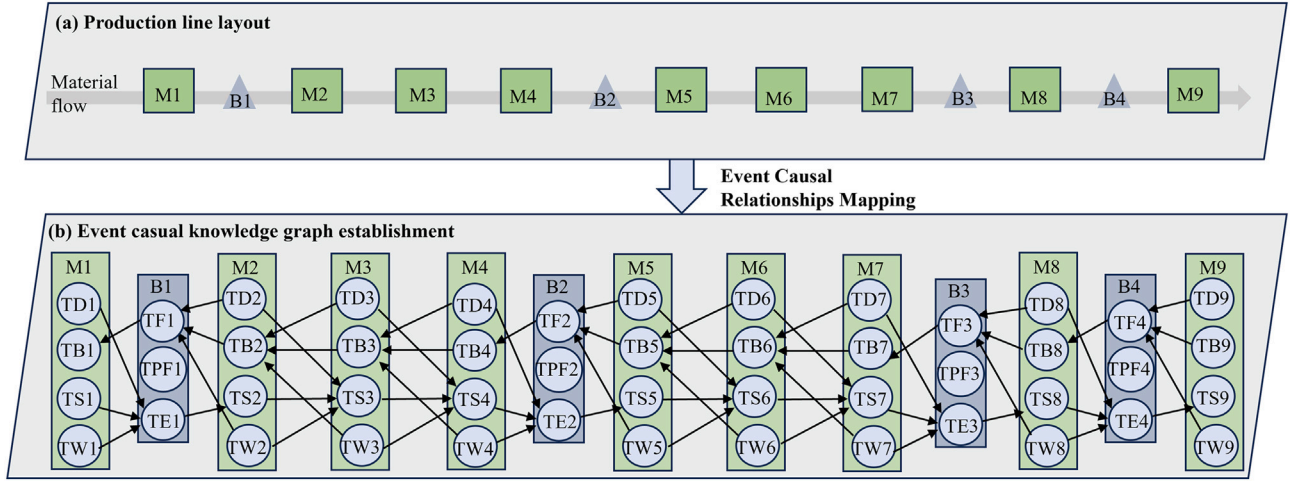


Fig. 2. Event causal graph mapping.

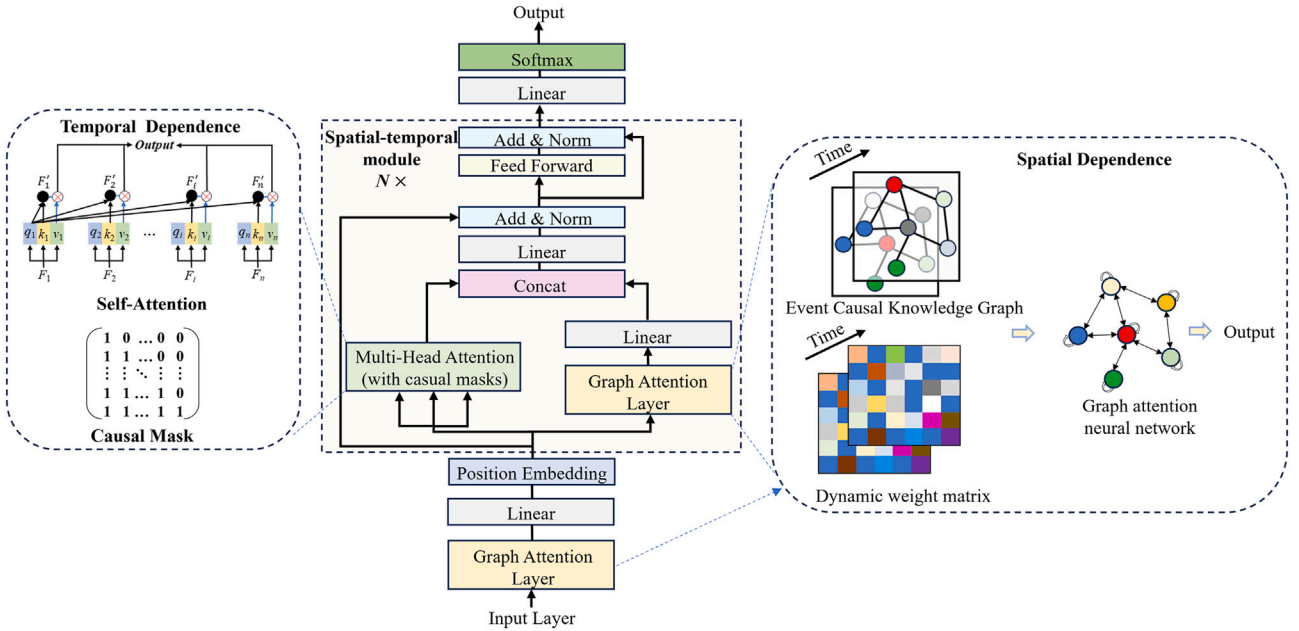


Fig. 3. The framework of the proposed model.

and spatial dimensional representations. The Spatial-Temporal Layer is designed to jointly model spatial correlations and temporal dynamics inherent in the event states of production systems, aiming to capture the interactions among different events (spatial dependencies) and the evolution of their operational statuses over time (temporal dependencies). Meanwhile, in the Spatial-Temporal Dependencies Interactive Fusion, spatial and temporal dependencies are jointly integrated to capture complex spatial-temporal patterns, enhancing the model's ability to predict key bottleneck indicators more accurately. The details of the proposed network are presented in Fig. 3. Finally, the predicted future blockage and starvation times are leveraged to identify bottlenecks using TPM (Fig. 4) in the production line.

3.3. Event causal graph establishment

The bottleneck shifts of a production process are affected by the integration of starvation and blockage time. Accurate prediction of future bottlenecks requires forecasting the two states. Since they are affected

by both station and buffer events (event features as section 3.1), an event causality graph is introduced to capture their causal relationships and dynamic interactions.

As shown in Eqs. (1) and (2), the events of a station are mutually exclusive in a period. Similarly, the states of a buffer are also mutually exclusive. Therefore, the interval events in a station or buffer have no causality relationships. As mentioned by [16], if the parts are not delivered to the downstream station or buffer in time, it may cause the downstream station to be starved (or the buffer to be empty), or the upstream station to be blocked (or the buffer to be full). This illustrates the causal relationships between station events and buffer events. According to expert knowledge, the down, operating, and starvation events of a station may lead to starvation in the downstream station (or an empty buffer). Similarly, the down, operating, and blocked events may cause blockage in the upstream station (or a full buffer). Based on these causal relationships, a event causality graph (ECA) is constructed, shown in Fig. 2. The ECA can be denoted as $\mathcal{G}$, as described in the Section 3.1. Since events are random, 1 and 0 are introduced to measure the weight

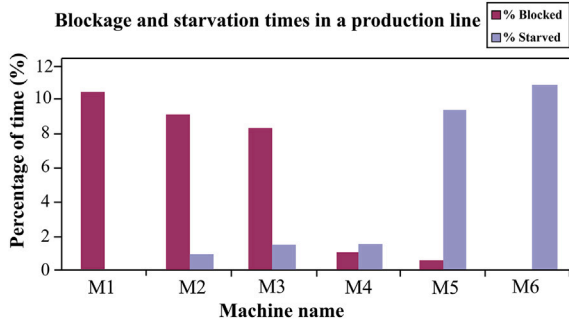


Fig. 4. Blockage and starvation times at time $t + T$ in a production line.

of the causal relationship between events to reflect the possibility of events and their causal influence over a defined time period.

Therefore, the adjacency matrix $\mathcal{A}$ of ECA can be defined as follows:

$$A_{ij,t} = \begin{cases} 1, & i \rightarrow j, \\ 0, & i \nrightarrow j. \end{cases} \quad (4)$$

where $A_{ij,t}$ denotes the weight between event i and j at a defined time period t . The $i \rightarrow j$ indicates a causal effect from i to j , while $i \nrightarrow j$ indicates its absence.

The graph structure is constructed during preprocessing and remains fixed during model training, while the weights are dynamically updated through temporal learning modules. The process of graph construction can be as follows:

Algorithm 1: The graph construction.

- 1 Extract event intervals from PLC logs, including machine ID, event type, start time, and end time.
 - 2 Aggregate each event duration within each time slot.
 - 3 Define candidate directed edges from production-flow logic and temporal precedence.
 - 4 $A_{ij,t} = 1$ only when event i is allowed by process logic to influence event j , and the corresponding temporal relation occurs within slot t or a predefined lag window.
 - 5 Use GAT attention to estimate the relative influence strength on the fixed candidate graph, yielding $W_{ij,t} = A_{ij,t} \alpha_{ij,t}$.
-

3.4. Network model

This section introduces the bottleneck architecture of the proposed ECGSTNN model, with the framework depicted in Fig. 3. In the input layer, the event features are first transformed into an event causal graph with a dynamic weight matrix, as described in Section 3.3. High-dimensional spatial features are extracted as the dataset passes through the Graph Attention Layer. Following positional embedding, long-term dependencies are captured by multi-head attention while short-term dependencies are extracted using the graph attention mechanism in the spatial-temporal module. Finally, bottleneck indicators are predicted by integrating spatial and temporal dependencies under constrained optimization.

The Graph Attention Network (GAT) effectively extracts features from graph-structured data by aggregating information from neighboring nodes to generate updated node representations. In the context of modeling random events, GAT's attention scores quantify causal relationships and spatial features in the event causal graph by assigning weights to edges based on their relevance. The GAT's attention mechanism is defined by the following formula:

$$e_{ij,t}^l = \text{LeakyReLU}(a^T(W_{i,t}^l h_{i,t}^{l-1} || W_{j,t}^l h_{j,t}^{l-1})), i, j \in \{1, 2, 3, \dots, N\} \quad (5)$$

where $W_{i,t}^l$ is a weight matrix to transform the initial features $h_{i,t}^l$ of each node into higher-level features. l denotes the l th layer of GAT. The vector a is the attention scores computed by the attention mechanism and $\cdot^T$ represent the transposition. The $(\cdot || \cdot)$ is the concatenation operation. LeakyReLU function is used as the nonlinearity. The value $e_{ij,t}$ is computed for nodes $j \in N_i$ in each period t , where N_i represents the neighborhood of node i in the graph. To make coefficients comparable across different nodes, the softmax function is leveraged to normalize the selection of all nodes j and can be expressed as follows:

$$\alpha_{ij,t}^l = \text{softmax}(e_{ij,t}^l) = \frac{\exp(e_{ij,t}^l)}{\sum_{u=1}^U \exp(e_{iu,t}^l)} \quad (6)$$

where u denotes the index of neighborhood of node i . The $\alpha_{ij,t}^l$ represents the normalized attention scores, which are used to computed combination of the embedding features for each node i to serve as the output of the l th GAT layer:

$$h_{i,t}^l = \sum_{u=1}^U A_{iu,t} \alpha_{iu,t} W_{u,t} h_{u,t}^{l-1} \quad (7)$$

where $A_{iu,t}$ is the dynamic constant of causal event relationships in period t computed according to Eq. (4).

To make the network more robust and more flexible in capturing the causal event relationships, the independent K multi-attention heads are used to compute different attention scores. Finally, the average function is leveraged to calculate the output embedding of the l th GAT layer with different attention heads, as follows:

$$\bar{h}_{i,t}^l = \frac{1}{K} \sum_{k=1}^K \sum_{j=1}^N A_{ij,t} \alpha_{ij,t}^k W_{j,t}^k h_{j,t}^{l-1} \quad (8)$$

Where $\bar{h}_{i,t}^l$ is the output of the GAT layer. The initial features of each node are transformed into higher-level embedding features using the GAT layers, enhancing expressiveness while increasing the network complexity and memory requirements. To address this, a linear function is applied to compress the embedding features of all nodes into lower-dimensional embeddings. Therefore, the initial inputs pass through the GAT to obtain the spatial dependencies, then a linear function transforms the output of the GAT layer, as follows:

$$X_e = \text{FC}(\text{GAT}(X_{in}, \mathcal{A})) = \text{FC}(\bar{h}_{i,t}^l) \quad (9)$$

Where FC represents a linear function. X_{in} is the initial input features and $\mathcal{A}$ denotes the adjacent weight matrix computed using Eq. (4).

Events exhibit temporal characteristics, as the evolution and state transitions of preceding events often influence current occurrences. Compared to recurrent neural networks (RNNs), Transformers enhance temporal modeling by injecting positional information using sinusoidal encoding, capturing global temporal dependencies through multi-head attention, and enforcing temporal causality with a causal mask. These components of sinusoidal position encoding, multi-head attention, and causal masking enable more effective learning of temporal patterns in event sequences and can be formulated as follows:

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/X_e}) \quad (10)$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos(\text{pos}/10000^{2i/X_e}) \quad (11)$$

$$X_{pe} = X_e + \text{PE}(X_e) \quad (12)$$

$$X_t = \text{Multihead}(X_{pe}) = \text{Concat}(\text{head}_1, \dots, \text{head}_n) W^O \quad (13)$$

where $\text{head}_i = \text{Attention}(Q_i, K_i, V_i) = \text{Softmax}(\frac{Q_i K_i^T}{\sqrt{d_{k_i}}}) V_i$. Q_i , K_i and V_i are the query, key and value of inputs and are projected by the learnable matrices W_i^Q , W_i^K and W_i^V . d_k is the dimension of K_i .

Although the multi-head attention mechanism can effectively model global temporal dependencies, it has limitations in its perception of local patterns. Graph Attention Network (GAT) can accurately capture local topological influences through neighborhood aggregation operations. Therefore, this study introduces the GAT module specifically for extracting spatial structural dependencies, shown as follows:

$$X_s = \text{GAT}(X_{pe}, \mathcal{A}) \quad (14)$$

where X_{pe} represents the output of the position embedding operation and $\mathcal{A}$ is the weight matrix.

We design a feature concatenation, linear transformation, and residual connection with a normalization module to effectively fuse and represent global and local dependencies. The concatenation integrates multi-scale contextual information from global and local dependencies, the linear layer maps high-dimensional features to promote interaction, and the residual connection with normalization preserves original information and standardizes feature distribution, enhancing the model's representation capability. The formula can be expressed as follows:

$$X_d = \text{LayerNorm}(X_{pe} + \text{FC}(X_t || X_s)) \quad (15)$$

where X_t and X_s represent the outputs of the multi-attention and GAT, respectively. $(\cdot || \cdot)$ denotes the concatenation operation.

To further enhance the deep dependency modeling between features, a feedforward layer is introduced to mine the potential nonlinear correlation between events through multi-layer linear transformations and nonlinear activation functions, improving the model's ability to perceive and predict future event states.

$$X_{p,out} = \text{FC}(\text{LayerNorm}(X_d + \text{FNN}(X_d))) \quad (16)$$

where $\text{FNN}(X_d) = \max(0, X_d W_1 + b_1) W_2 + b_2$. X_d is the output of global and local information concatenation. W_1 and W_2 represent the learnable matrices. b_1 and b_2 are bias parameters. FC is a linear function.

Finally, due to the constraint relationships among events as Eqs. (1) and (2), we apply the Softmax function to normalize equipment and buffer events, ensuring that their states satisfy mutual exclusivity constraints.

$$\hat{X}_{t+1} = \text{Softmax}(X_{p,out}) \quad (17)$$

During the model training, the error between predicted values and true values is computed by the Mean Squared Error (MSE) function overall training samples. It measures the overall quality of the model's predictions, with lower MSE indicating better performance. The AdamW optimizer with a learning rate λ is employed to update the model parameters based on the computed gradients, facilitating the optimization process. The MSE loss function is expressed as follows:

$$\text{Loss} = \frac{1}{n} \sum_{t=1}^n (X_{t+1} - \hat{X}_{t+1})^2 \quad (18)$$

3.5. Bottleneck identification and prediction

Many data-driven methods have been employed for throughput bottleneck identification, including the arrow-based method [46], active period method [47], maximum production load method [48], and turning point method (TPM) [14,27]. Unlike approaches focused on system optimality, TPM is a data-driven approach that captures system performance over both short and long-term horizons. Therefore, this paper adopts TPM as the method for bottleneck identification. The method leverages blockage and starvation time as indicators for bottleneck identification. During time $t + T$, bottlenecks can be identified when blockage

Algorithm 2: Bottleneck prediction.

Input: Initial feature input $X_{in} = [X_{t-T_s+1}, \dots, X_{t-1}, X_t] \in \mathbb{R}^{N \times T_s}$ and adjacent matrix $\mathcal{A} \in \mathbb{R}^{N \times N \times T_s}$;

Output: The bottleneck machines M_b ;

- 1 // Normalize the initial inputs X_{in} .
- 2 $X_{in} \leftarrow \text{Norm}(X_{in})$;
- 3 // The initial features are first fed into the GAT module to obtain the spatial embedding, and then enter the multi-attention module of transformer and the GAT module to capture the long-term and short-term dependencies in parallel, which are concatenated for the following output operation.
- 4 **Function** *ECGBSTNN* ($I_{in}, \mathcal{A}$):
- 5 $X_e \leftarrow \text{FC}(\text{GAT}(X_{in}, \mathcal{A})) = \text{FC}(\bar{h}_{i,t}^l)$
- 6 // From the Eqs. (10)–(13), the long-term dependencies of features can be obtained.
- 7 $X_{pe} \leftarrow X_e + \text{PE}(X_e)$;
- 8 $X_t \leftarrow \text{Multihead}(X_{pe})$;
- 9 // From the Eqs. (5)–(8) and Eq. (14) capture the short-term dependencies.
- 10 $X_s \leftarrow \text{GAT}(X_{pe}, \mathcal{A})$;
- 11 // The output operations are as Eqs. (15)–(17).
- 12 $X_d \leftarrow \text{LayerNorm}(X_{pe} + \text{FC}(X_t || X_s))$;
- 13 $X_{p,out} \leftarrow \text{FC}(\text{LayerNorm}(X_d + \text{FNN}(X_d)))$;
- 14 $\hat{X}_{t+1} \leftarrow \text{Softmax}(X_{p,out})$;
- 15 // The future starvation time (TS_f) and blockage time (TB_f) are extracted from $\hat{X}_{t+1}$.
- 16 $TS_f, TB_f \leftarrow \hat{X}_{t+1}$;
- 17 **return** TS_f, TB_f .
- 18 **end**
- 19 // Output the bottleneck machine M_b identified by TPM with starvation and blockage time.
- 20 $M_b \leftarrow \text{TPM}(TS_f, TB_f)$.

and starvation time meet the following conditions:

if $j \neq 1, j \neq N$:

$$TB_{i,t+T} - TS_{i,t+T} > 0 : i \in [1, \dots, j-1],$$

$$TB_{i,t+T} - TS_{i,t+T} < 0 : i \in [j-1, \dots, N],$$

$$TB_{j,t+T} - TS_{j,t+T} < TB_{j-1,t+T} - TS_{j-1,t+T},$$

$$TB_{j,t+T} - TS_{j,t+T} < TB_{j+1,t+T} - TS_{j+1,t+T}. \quad (19)$$

if $j = 1$:

$$TB_{1,t+T} - TS_{1,t+T} > 0,$$

$$TB_{2,t+T} - TS_{2,t+T} < 0,$$

$$TB_{1,t+T} - TS_{1,t+T} < TB_{2,t+T} - TS_{2,t+T}. \quad (20)$$

if $j = N$:

$$TB_{N-1,t+T} - TS_{N-1,t+T} > 0,$$

$$TB_{N,t+T} - TS_{N,t+T} < 0,$$

$$TB_{N,t+T} - TS_{N,t+T} < TB_{N-1,t+T} - TS_{N-1,t+T}. \quad (21)$$

where TB and TS denote the blockage and starvation times, respectively. N is the total number of machines, j is the bottleneck station and T denotes a period.

For example, based on the above Eqs. (19)–(21) and the results shown in Fig. 4, it can be clearly observed that the bottleneck is at station M_4 in a production line. Bottleneck identification relies on key indicators. Therefore, bottleneck prediction can be viewed as forecasting the future distributions of blockage and starvation to identify potential future bottleneck locations using the turning point method.

Table 1
The example of events recorded in the production line.

Machine	Start time	End time	Event
M ₁	15:13:32 05-12-2025	15:14:18 05-12-2025	oprating
M ₁	15:14:18 05-12-2025	15:14:42 05-12-2025	down
...	...	...	...
B ₁	15:13:32 05-12-2025	15:14:58 05-12-2025	Partially Full

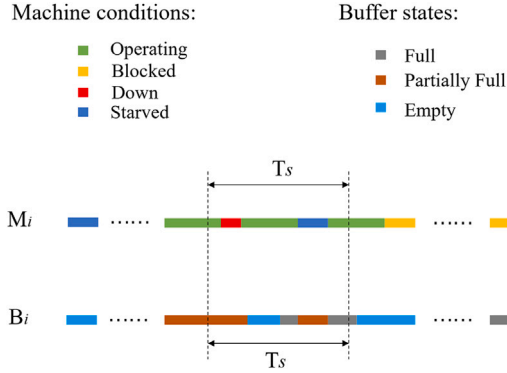


Fig. 5. The example of event time statistics.

4. Experiment

4.1. Experiment establishment

The dataset is collected from a discrete production line, as shown in Fig. 2(a), capturing station states and buffer conditions in the log data (Table 1). These are transformed into corresponding features representing operating, blocked, starved, and down states, as well as full, partially full, and empty buffer times. As events in the production line occur sequentially, their respective durations are calculated over a period, as illustrated in Fig. 5. Moreover, the causal weights of sequential events are computed using the Eq. (4) in different periods across various time intervals to quantify their dynamic relationships. The sliding window technique is utilized to divide the dataset, facilitating one-step-ahead predictions based on historical data. Finally, based on the bottleneck metrics of each station, the future starvation times and blockage times are employed to identify the bottlenecks.

Due to the differences in numerical scales of different features, the model may assign higher weights to features with a larger value range during training, leading to an unstable optimization process or slow convergence. Normalization can make each feature fall within the same or similar scale range, which not only helps to improve the stability and efficiency of model training, but also facilitates subsequent analysis such as feature selection and importance evaluation. The minimum-maximum normalization function is used to normalize all features to a value range of 0 to 1. The function can be expressed as follows:

$$\hat{x}_t^{a,b} = \frac{x_t^{a,b} - x_{min}^a}{x_{max}^b - x_{min}^b} \quad (22)$$

where $x_t^{a,b}$ is the value of the a^{th} row and the b^{th} column of data at time t , x_{max}^b and x_{min}^b represent the maximum and minimum values in the c^{th} column of data, respectively.

The key hyperparameters applied for model training are summarized in the Table 2. The model training parameters mainly include learning rate, optimizer, layer, batch size and hidden size. The AdamW optimizer is used for model training, and the initial learning rate is set to 0.0001, so that the model parameters converge smoothly in the optimization

Table 2
Hyperparameters for experimental study.

Hyperparameters	Value/method
Learning rate λ	0.0001
Optimizer	AdamW
GAT layer	1
Temporal layer	4
Batchsize	64
GAT hidden size units	64
Temporal hidden size units	256
Heads of multi-head attention	8

Table 3
Definition of event nodes and corresponding IDs.

Machine	TW	TB	TS	TD	Buffer	TPF	TF	TE
M1	e1	e2	e3	e4	B1	e49	e41	e42
M2	e5	e6	e7	e8	B2	e50	e43	e44
M3	e9	e10	e11	e12	B3	e51	e45	e46
M4	e13	e14	e15	e16	B4	e52	e47	e48
M5	e17	e18	e19	e20	-	-	-	-
M6	e21	e22	e23	e24	-	-	-	-
M7	e25	e26	e27	e28	-	-	-	-
M8	e29	e30	e31	e32	-	-	-	-
M9	e33	e34	e35	e36	-	-	-	-
M10	e37	e38	e39	e40	-	-	-	-

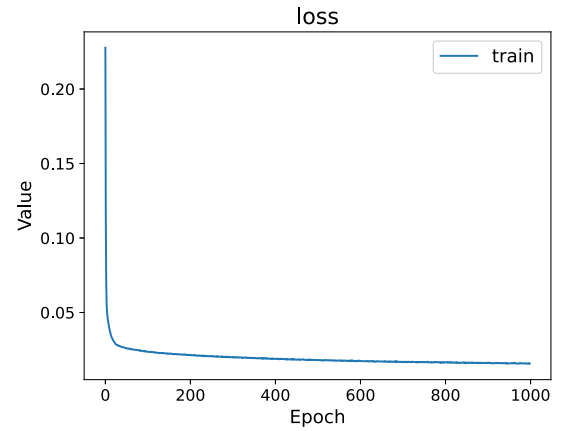


Fig. 6. Trend of the loss function over training epochs.

space and finally reach a stable state. The proposed model is configured with a single-layer GAT for spatial representation learning and four temporal layers for sequence modeling. The training batch size is 64. The GAT layer comprises 64 hidden units, and each temporal layer includes 256 hidden units. Furthermore, the multi-head attention mechanism employs 8 parallel attention heads. Besides, the definitions of the event nodes and their corresponding IDs are summarized in Table 3. The dataset is split for training (75%), testing (15%) and validation (15%). The experiments are conducted on a machine equipped with an Intel(R) Core(TM) i7-9700 CPU and 16 GB of RAM.

During the model training process, the evolution of the loss function over epochs provides critical insight into the optimization dynamics and convergence behavior of the proposed ECGBSTNN model. As illustrated in Fig. 6, the training loss typically undergoes three distinct phases. In the initial phase (approximately epochs 0 to 30), the loss decreases rapidly, indicating that the model is quickly learning basic patterns and adjusting weights effectively. This is followed by a gradual optimization phase (around epochs 30 to 800), during which the rate of loss reduction slows down, reflecting fine-tuning of parameters and deeper pattern extraction. Finally, after epoch 800, the loss stabilizes and converges to a relatively steady value, suggesting that the model has reached an

optimal state with minimal further improvements. The three-stage trend demonstrates the effectiveness of the optimization and the model's stable convergence.

After model training, the blockage time and starvation time of each station in the future period $t+1$ can be predicted. The model performance is evaluated by calculating mean absolute error (MAE) and root mean squared error (RMSE) between the predicted values of blockage time and starvation time and the true values. The calculations of MAE and RMSE can be expressed as follows:

$$MAE = \frac{1}{n} \sum_{i=1}^n |X_{t+1} - \hat{X}_{t+1}| \quad (23)$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (X_{t+1} - \hat{X}_{t+1})^2} \quad (24)$$

where X_{t+1} , and $\hat{X}_{t+1}$ denote the ground truth, and the predicted values at time $t+1$, respectively. The variable n represents the total number of data samples.

4.2. Experiment results and discussions

4.2.1. Experiment results

In the study, the TPM method is adopted to detect bottlenecks within the production line. The blockage and starvation time, as its core indicators, are forecasted to support proactive identification of future bottlenecks. To evaluate the prediction performance of the proposed method, assessments are conducted on M1, M6, and the overall production line. Since M1 is located at the input end of the production line, it is less likely to experience starvation but may frequently encounter blockage due to disturbances in downstream processing. This makes it a suitable candidate for observing blockage patterns and assessing the causal modeling capability. In contrast, M6 is situated between M5 and M7 without an intermediate buffer, making it highly susceptible to becoming a bottleneck. Therefore, M1 is selected as a representative station for evaluating blockage prediction performance, while M6 serves to assess the model's ability to capture bottleneck tendencies. M1 rarely experiences starvation as there are no upstream buffers or machines that can cause supply interruptions. Since the ground truth for M1 starvation is nearly zero, most models naturally yield a zero prediction error as shown in the results of M1 in the Table 4.

To evaluate the effectiveness of the proposed method (ECGBSTNN), five existing methods, namely LSTM [32], Transformer [25], BSTAN [15], BTGCN [20], and BPSTK [21], are employed as benchmarks for predicting blockage and starvation time. The prediction performance is assessed using two metrics (MAE and RMSE), evaluated on stations M1, M6, and the overall production line. As listed in the Table 4,

Table 4

The performance comparison of different methods evaluated on M1, M6 and the overall production line.

Indicator	Method	M1		M6		Overall	
		MAE	RMSE	MAE	RMSE	MAE	RMSE
Starvation	LSTM [32]	0.00	0.00	27.04	44.64	19.07	36.12
	Transformer [25]	0.02	0.03	23.04	38.69	15.68	34.85
	BSTAN [15]	2.96	9.80	24.73	41.58	18.15	34.98
	BTGCN[20]	1.76	2.55	24.28	39.03	17.62	33.25
	BPSTK[21]	0.00	0.00	20.88	36.79	17.04	35.22
	ECGBSTNN	0.00	0.00	10.58	30.17	9.38	24.99
Blockage	LSTM [32]	33.81	53.10	21.20	31.30	16.09	30.57
	Transformer [25]	29.69	54.33	15.65	25.91	11.83	26.64
	BSTAN [15]	27.17	41.13	26.95	49.93	13.58	29.67
	BTGCN[20]	28.31	39.59	27.71	32.61	14.64	26.58
	BPSTK[21]	26.42	42.46	24.13	45.68	12.45	27.09
	ECGBSTNN	17.05	32.37	13.50	25.26	9.10	23.65

the experimental results demonstrate that the proposed ECGBSTNN model consistently outperforms the baseline methods in predicting both blockage and starvation times, particularly in terms of MAE.

As shown in Table 4, the LSTM underperforms compared to the GNN-GRU integrated approaches, including BSTAN, BTGCN, and BPSTK. This indicates that GNNs can effectively capture spatial relationships and leverage spatiotemporal dependencies to enhance prediction performance. Although both BSTAN and BPSTK are based on GAT-GRU architectures, they differ in how they incorporate spatiotemporal dependencies. BPSTK leverages a knowledge graph to explicitly encode domain relationships before inputting data into the model, enabling structured semantic guidance. In contrast, BSTAN relies on the model itself to implicitly learn such dependencies from raw inputs. The BPSTK method performs better than the BSTAN method. This comparison highlights that incorporating prior relational information through data preprocessing can enhance model performance. In addition, the Transformer outperforms the LSTM, indicating that the multi-head attention mechanism effectively captures contextual dependencies and long-term temporal relationships among workshop events, especially those caused by data latency, thereby enhancing model performance. The GAT-based BSTAN and BPSTK methods achieve lower MAE than the GCN-based BTGCN method, indicating that the models yield relatively small average prediction errors and stable performance. Based on GAT learning spatial relations, data feature relationship preprocessing and the multi-head attention mechanism to improve model capabilities, the ECGBSTNN model is proposed. The model performs well in predicting blockage and starvation times, especially achieving reductions of 2.73 and 2.93 in MAE and RMSE of the overall production line, respectively, for blockage time across the entire production line compared to the best baseline model (Table 4).

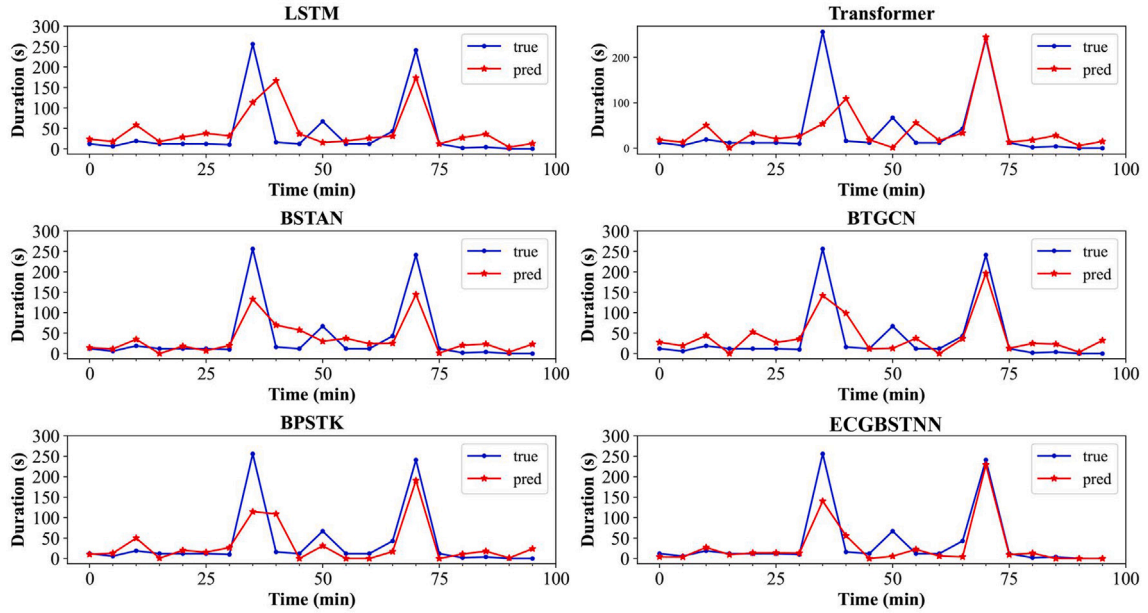
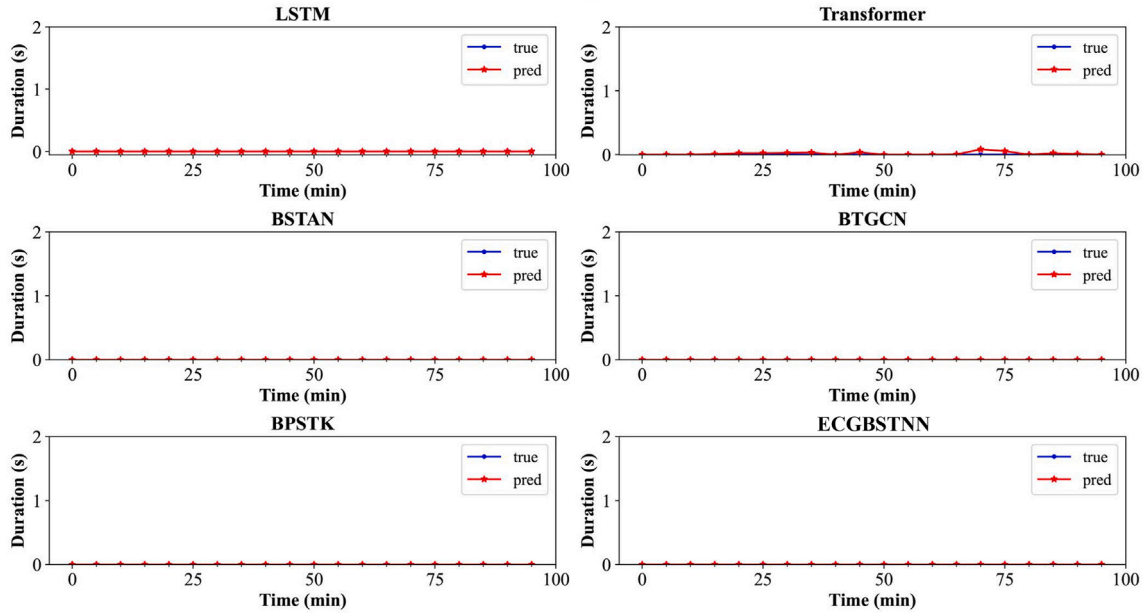
For analytical purposes, the predicted blockage and starvation time for M1 and M6 are plotted in Figs. 7 and 8, respectively. In a normal case, M1 is located at the beginning of the production line and rarely experiences starvation due to sufficient incoming material. However, it may be blocked by downstream disturbances. As a result, the starvation prediction curve mostly remains at zero, showing high prediction accuracy. This indicates that the starvation time prediction of M1 is primarily driven by its temporal dynamics, with minimal influence from other station events, as reflected in the causal event graph modeling shown in Fig. 2. The blockage time of M1 and M6, and the starvation time of M6, are influenced by upstream and downstream events, reflecting clear causal and spatial-temporal dependencies. The proposed method is capable of effectively capturing these interdependent patterns with the causal event graph modeling. As a result, it demonstrates superior performance in predicting the evolving states of blockage and starvation, as shown in Figs. 7 and 8. However, at 50min, the predicted value is much lower than the true value. It may be interrupted by the outside environmental factors, which were recorded in historical data. Therefore, it is not well predicted.

In the Table 5, the result of complexity analysis comparison shows that Floating-point operations (FLOPs) and the number of parameters (Params) are positively correlated with the dimensionality of hidden layers. As the hidden dimension increases, the model complexity correspondingly grows. Therefore, simpler networks tend to have fewer parameters and lower computational costs, whereas more complex architectures exhibit higher values. For example, an LSTM model has only 4.232M FLOPs and 132.648K parameters. In ECGBSTNN, the GAT hidden layer configuration is [64, 16, 52, 64]. Compared with BSTAN and BPSTK, which use [64, 16, 10, 64], ECGBSTNN results in higher FLOPs but fewer parameters. This is because ECGBSTNN uses the number of event features (52) as the node count, whereas BSTAN and BPSTK use the number of machines (10) as the node count. Consequently, the increase in node count leads to higher computational costs. In ECGBSTNN, high-dimensional machine-level features are decomposed into multiple low-dimensional event-level features, reducing the size of weight

Table 5

The complexity analysis among different methods.

Method [Batch size, Sequence length, Hidden size]	Flops	Parameters (Params)	Training time for a batch (s)
LSTM [64, 16, 64]	4.232M	132.648K	0.238
LSTM [64,16, 256]	60.969M	1.907M	0.258
Transformer [64,16, 64]	33.833M	1.065M	0.252
Transformer [64,16, 64]	135.332M	4.235M	0.286
BSTAN [64,16, (GAT: 10, 64) / (GRU:64)]	97.632M	344.104K	0.247
BSTAN [64,16, (GAT: 10, 64) / (GRU:256)]	204.931M	2.015M	0.255
BPSTK [64, 16, (GAT: 10, 64) / (GRU: 64)]	99.271M	345.744K	0.251
BPSTK [64, 16, (GAT: 10, 64) / (GRU:256)]	206.569M	2.017M	0.260
ECGBSTNN [64, 16, (GAT: 52, 64) / (Transformer:64)]	3.075G	3.011M	0.595
ECGBSTNN [64, 16, (GAT: 52, 64) / (Transformer:256)]	6.793G	6.637M	0.671

The blockage time prediction at M1**The starvation time prediction at M1****Fig. 7.** The blockage and starvation time predictions at M1.

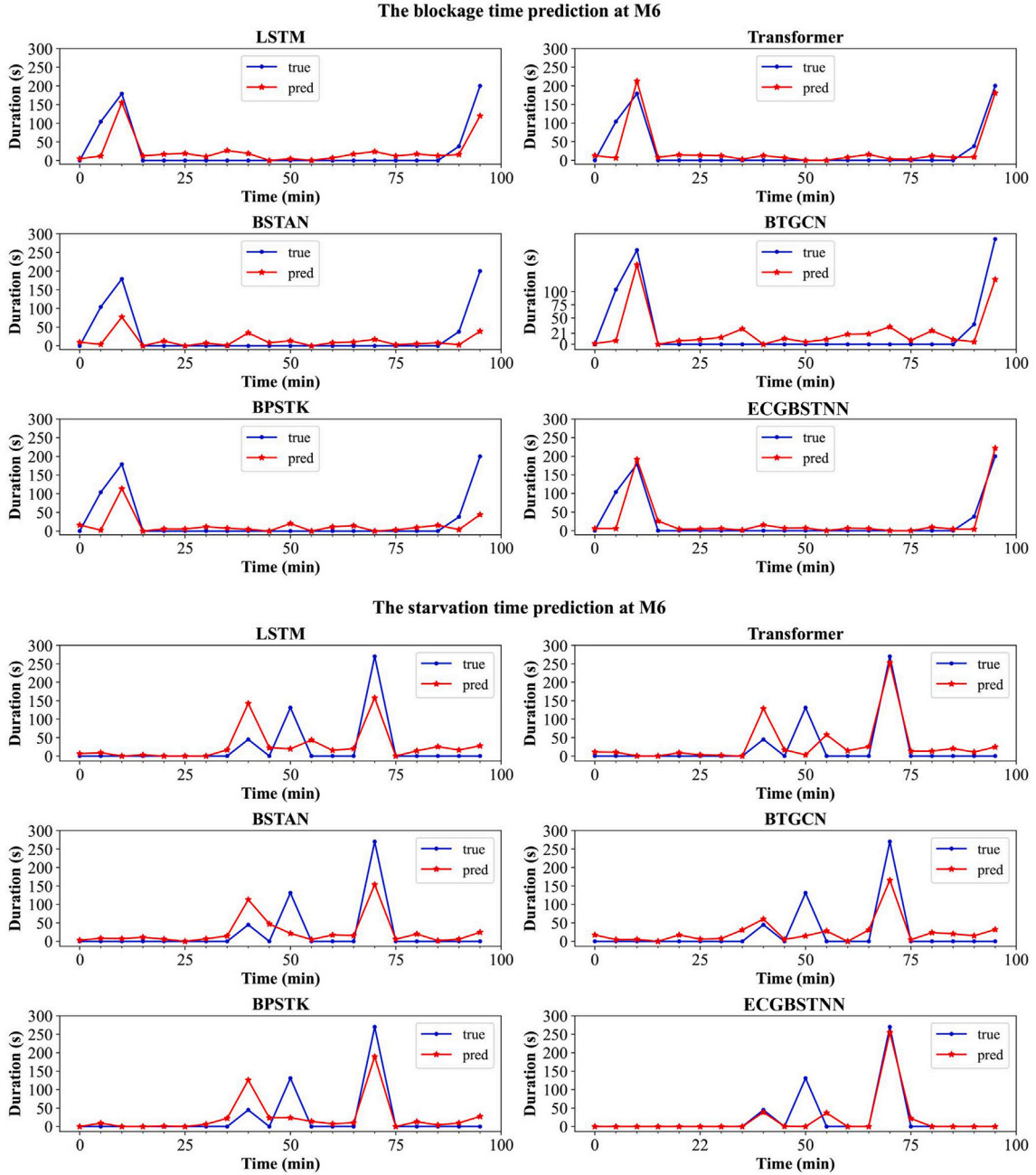


Fig. 8. The blockage and starvation time predictions at M6.

matrices. Moreover, parameters in the GAT layers are shared across nodes and do not scale with node count. Training time increases with FLOPs but exhibits a clear sublinear trend, where a substantial increase in FLOPs results in only a moderate increase in training time. In contrast, the number of parameters shows little correlation with training time. For example, Transformer [64,16,256] contains more parameters than ECGBSTNN (Transformer 64) but achieves faster execution. This indicates that training time is primarily influenced by computational complexity and model architecture (e.g., parallelism and dense versus sparse operations), rather than parameter count alone.

To evaluate the contribution of each component to the overall performance of ECGBSTNN, some ablation experiments isolate specific components of the model to assess their impact on performance, providing insights into their roles in capturing causal and temporal dynamics. The experiments are described as follows:

1. GAT + Transformer: In this variant, input features are first processed using a GAT to aggregate the influence of causal events, followed by a Transformer that captures temporal dynamics through contextual dependency learning. Unlike ECGBSTNN, which models short-term and long-term dependencies in a parallel model as shown in Fig. 3, this lacks the graph attention mechanism for short-term dependencies.
2. ECGBSTNN(gcn): The GAT modules in the proposed method are replaced with the GCN modules, which use fixed weights in each period instead of dynamic attention scores, potentially limiting complex causal relationship modeling. GAT prioritizes influential events dynamically, whereas GCN aggregates neighbors statically.
3. SGBSTNN: The edges represent material flows between stations, and the weights reflect process-related relationships, not event

causality. The algorithmic framework remains unchanged (seen in Fig. 3). The modeling has only shifted from event-causal graphs to graphs representing station process associations with event features. The station graph, with features in each station, is fed into the algorithm to predict starvation and blockage time, enabling future bottleneck identification.

As shown in the Table 6, the GAT + Transformer model outperforms the Transformer, indicating that GAT effectively captures the relationships among events in the causal graph and improves

Table 6

The performance evaluation of ablation experiments on the overall production line.

Indicator	Method	MAE	RMSE
Starvation	Transformer	15.68	34.85
	GAT + Transformer	13.05	26.77
	ECGBSTNN (gcN)	13.07	28.22
	SGBSTNN	12.72	27.54
	ECGBSTNN	9.38	24.99
Blockage	Transformer	11.83	26.64
	GAT + Transformer	11.73	26.12
	ECGBSTNN (gcN)	11.31	28.22
	SGBSTNN	10.48	23.51
	ECGBSTNN	9.10	23.65

prediction performance. Compared to GAT + Transformer, the proposed ECGBSTNN further integrates a parallel GAT module and multi-head attention mechanism. While the GAT focuses on capturing short-term dependencies, the attention mechanism captures long-term dependencies, and their fusion leads to a more comprehensive representation. The architecture significantly improves model performance, especially achieving a 2.63 reduction in MAE for blockage prediction compared to the GAT + Transformer method. Compared to ECGBSTNN (GCN), the proposed method also performs better, which indicates that the GAT can learn the spatial dependencies better than the GCN through the suitable and dynamic attention scores for the edge weights in the event causal graph during different periods. Moreover, to evaluate the event-causal graph modeling, the proposed model is compared to the SGBSTNN, and it outperforms the latter, especially in starvation, with a 3.34 reduction in MAE and a 2.55 reduction in RMSE.

To demonstrate how the machine-level graph and the event causal graph differently and adaptively adjust edge weights during the GAT learning process, and to emphasize the enhanced interpretability of the event causal graph, we plotted heatmaps of the attention scores generated by the graph self-attention mechanism for both models, as shown in Fig. 9. In Fig. 9(a) and (b), M1 to M10 represent the machines, and the attention scores indicate the weights of connections between these machines. By comparing Fig. 9(a) and (b), it is evident that while attention scores fluctuate over different periods, a consistent correlation pattern among stations is preserved throughout the learning process. This suggests the presence of stable inter-feature dependencies between stations.

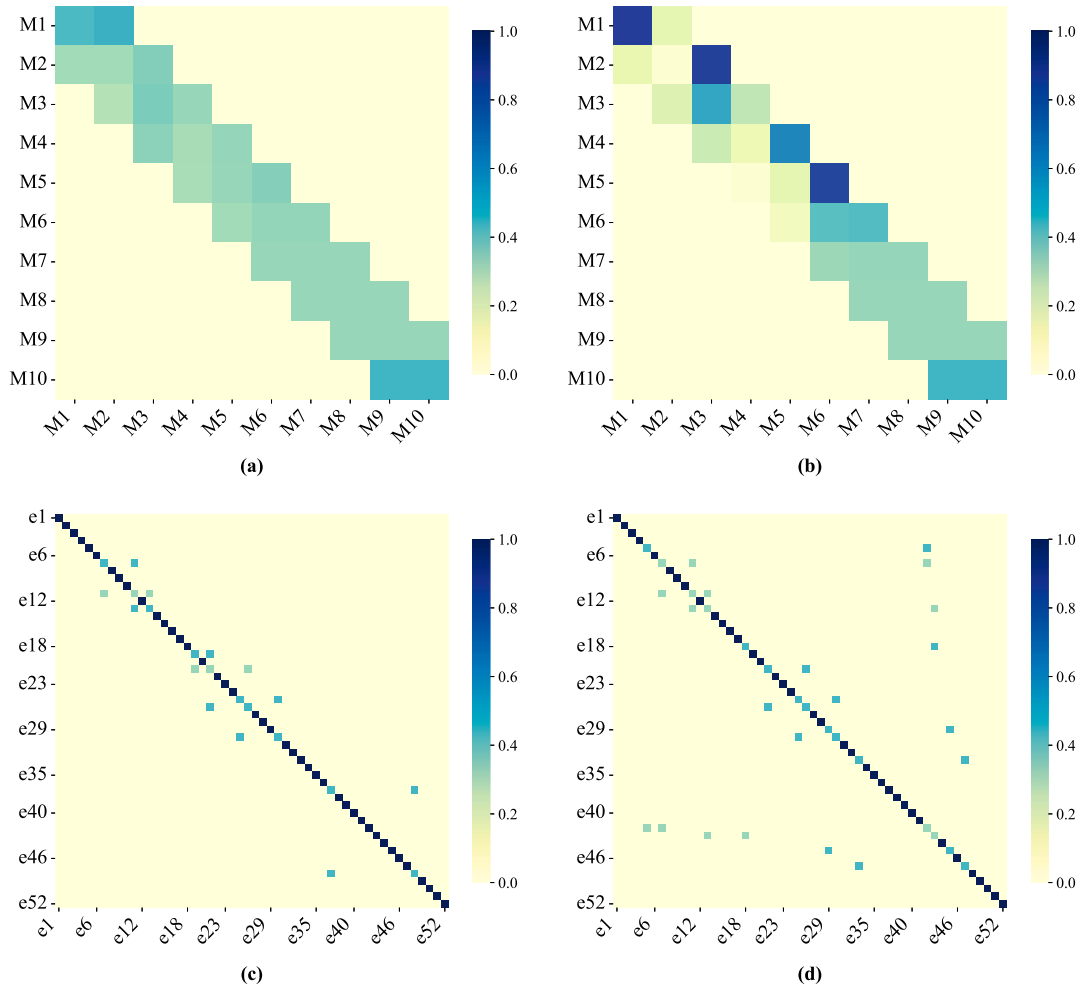


Fig. 9. The attention scores of two models at different periods. (a) the first period of SGBSTNN; (b) the last period of SGBSTNN; (c) the first period of ECGBSTNN; (d) the last period of ECGBSTNN.

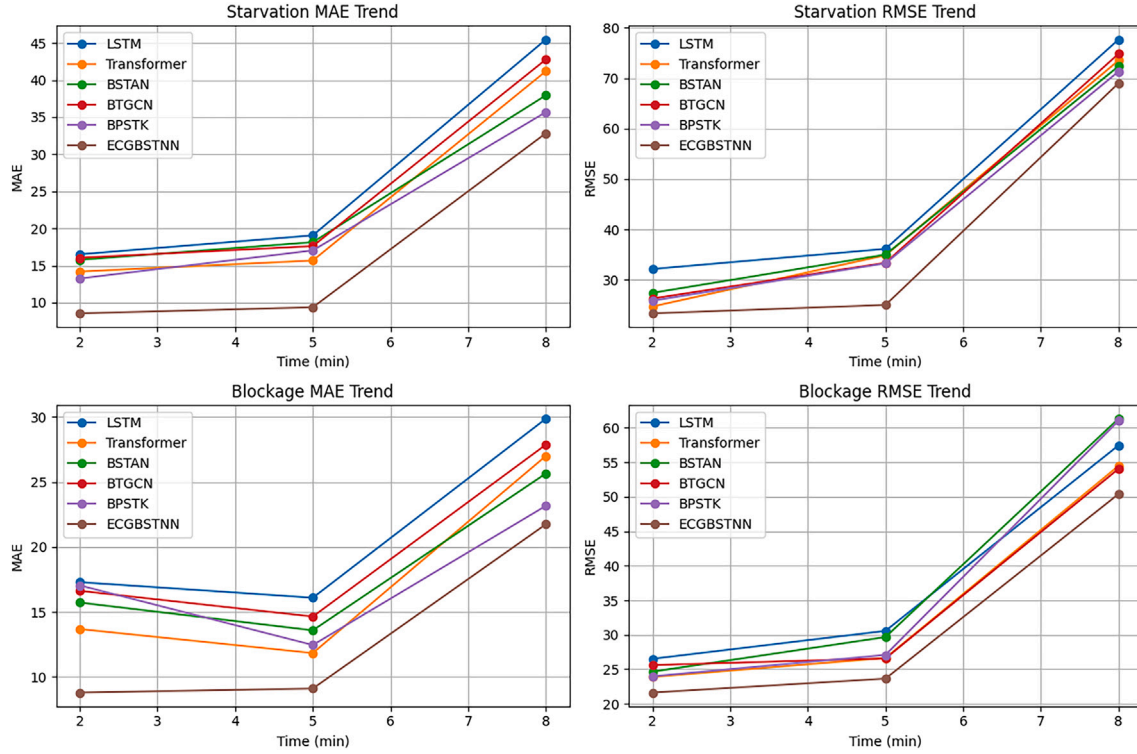


Fig. 10. Robustness analysis evaluates the performance of bottleneck prediction across the production line for different time-slot sizes.

To further highlight the dynamic adjustments within the event causality modeling, we generated two additional heatmaps illustrating the causal weights of events across different periods, as presented in Fig. 9(c) and (d). Each period involves multiple event types occurring at various locations: four event types at each station and three event types at each buffer. These events influence each other through causal effects, with their weights varying over time. Here, e1 to e52 denote unique events at different stations and buffers. The heatmap on the left corresponds to the initial period, while the one on the right represents the final period. The changes in color patterns between these two heatmaps reveal the evolving causal relationships during training. For instance, the relationship between e7 and e42 can be more clearly understood by combining the information presented in Table 1, Figs. 1 and 2. Event e42 corresponds to the empty state of buffer B1. When B1 experiences an empty state, material does not flow into the downstream, which causes machine M2 to reach e7 (TS2). Similarly, event e7 may further propagate downstream and cause starvation at machine M3, which is represented by event e11 (TS3). This indicates that events follow a temporal causal sequence and their influence on other events varies across different time periods. Additionally, a blue highlight on an event within a specific period indicates an attention score of 1, meaning that the event's state primarily depends on its own features and is not significantly affected by other events during that period. These visualizations vividly capture the temporal evolution of event-level causal dependencies, providing a solid basis for interpreting the underlying causal mechanisms. The observed dynamic shifts in attention scores effectively reflect the changing influence among events over time. Compared to the graph based on stations, the event causal graph offers a more detailed and granular representation of relationships, thus enhancing explainability. By capturing the fine-grained and dynamic interactions between individual events, the event causal graph allows the model to better understand complex temporal dependencies. As a result, the ECGBSTNN framework, which is built on the event causal graph, demonstrates superior ability in capturing causal structures and delivers more accurate

predictions than the station-based SGBSTNN model, while the one on the right represents the final period. The changes in color patterns between these two heatmaps reveal the evolving causal relationships during training.

To evaluate the robustness of the proposed method, we conducted comparative experiments using datasets aggregated over different timeslot sizes including 2-minute, 5-minute, and 8-minute intervals. These datasets were employed to predict blockage and starvation durations. The results reveal a gradual decline in performance as the time interval lengthens, which is likely due to the reduction in temporal resolution that can obscure important data patterns. Despite this, the proposed method consistently outperforms baseline approaches, achieving higher accuracy and greater stability, as reflected by MAE and RMSE metrics presented in Fig. 10. This outcome indicates that the method effectively captures the essential dynamics of the manufacturing process, even when temporal granularity decreases. Furthermore, its adaptability across different time scales highlights its practical value in real-world applications, where data collection intervals may vary due to operational constraints or system limitations. Importantly, the proposed approach demonstrates lower sensitivity to increases in timeslot size, whereas benchmark models exhibit more significant performance degradation, further underscoring its robustness and reliability in diverse temporal settings.

Besides, to further show the generalizability of the proposed approach, another scenario is leveraged to compare the ability of different methods. In Fig. 3, Scenario 2 was used in previous research [33] to generate the dataset. In the table, the results demonstrate that ECGBSTNN consistently achieves the best performance across all scenarios and tasks. Compared with the best baseline, it significantly reduces both MAE and RMSE, particularly in Scenario 1. Moreover, although Scenario 2 introduces more complex conditions, ECGBSTNN still maintains clear performance advantages, while other methods show noticeable degradation. This stability across different scenarios highlights the strong generalization capability of the proposed model (Table 7).

Table 7
The performance of different models in two different scenarios.

Scenario	Indicator	Overall	Method					
			LSTM	Transformer	BSTAN	BTGCN	BPSTK	ECGBSTNN
Scenario 1	Starvation	MAE	19.07	15.68	18.15	17.62	17.04	9.38
		RMSE	36.12	34.85	34.98	33.25	35.22	24.99
	Blockage	MAE	16.09	11.83	13.58	14.64	12.45	9.10
		RMSE	30.57	26.64	29.67	26.58	27.09	23.65
Scenario 2	Starvation	MAE	30.25	14.68	16.69	16.93	13.81	12.53
		RMSE	29.54	20.03	21.63	21.27	17.32	14.87
	Blockage	MAE	26.68	20.56	21.68	22.08	19.56	17.43
		RMSE	35.12	24.35	25.98	25.51	23.76	21.79

4.2.2. Discussions

The experimental results show that the proposed ECGBSTNN framework achieves better predictive performance than existing leading methods, as reflected by lower MAE and RMSE values. This improvement comes from preprocessing event feature relationships to make the model more interpretable, using a directed and weighted causal graph to capture dependencies between events, and combining multi-head attention with Graph Attention Networks (GAT) to model both long-term and short-term temporal patterns. Together, these elements allow the framework to represent the changing dynamics of manufacturing processes with high accuracy.

In real production settings, each operational unit (e.g., stations, buffers, operators) can be represented as event states reflecting its working condition. These events exhibit natural causal links that encode the underlying process logic. By comparing the process graph constructed at the station level with the event causal graph built at the feature level, it is observed that in the station graph, edges between nodes influence each other at every time step. This may result from certain features of one station being related to features of other stations, though such relationships are not explicitly apparent in the graph structure. In contrast, the event causal graph, based on event features, captures direct causal effects between events and reveals dynamic associations among event features. These dynamic changes can be clearly seen through the attention scores in the graph. By directly modeling these causal links, ECGBSTNN enhances the interpretability of system behavior and provides valuable insights for data-driven bottleneck prediction. Due to the persistence of events, delayed effects occur, causing events to exhibit not only short-term but also long-term temporal dependencies. Therefore, we incorporate a Transformer module to capture long-term temporal dependencies and compare it with traditional RNN models such as LSTM, achieving superior results. Additionally, we design a parallel long and short-term temporal dependency module, which further improves the model's performance. This capability is particularly beneficial for production planning and maintenance, where accurate forecasting of bottlenecks and inefficiencies can lead to increased efficiency and reduced downtime.

Regarding the reproducibility of the proposed algorithm, the event causal graph is first constructed based on the production line process flow, as shown in Fig. 2. Then, using the event sequences from the log data (Table 1) and applying Eq. (4), the graph edges are dynamically updated to capture evolving relationships. Event durations within each cycle are also recorded, illustrated in Fig. 5, resulting in the final feature-level event causal graph. With algorithm parameters set as in Table 2, Algorithm 2 computes the future starvation and blockage distributions, which are subsequently used by the TPM method to identify potential bottlenecks. This stepwise procedure provides a clear pathway to reproduce the algorithm. However, due to data confidentiality agreements with industry partners, direct access to the original datasets is restricted, limiting the exact replication of the experimental results by external researchers.

While the current study applies this approach to a linear production line, the underlying event causal graph framework is highly adaptable

and can be extended to more complex nonlinear discrete manufacturing systems, where processes may run in parallel, loop back, or branch out. By enriching node and edge definitions, the framework can capture intricate relationships and feedback loops, enabling effective modeling of complex causal dynamics in real-world manufacturing environments. However, as the complexity and scale of these systems increase, the graph structures grow larger and more computationally demanding. To address these scalability challenges, techniques such as graph partitioning, hierarchical graph construction, pruning, distillation and mini-batch sampling can be applied to reduce computational overhead while preserving critical causal information. The incorporation of sparse attention mechanisms further enhances efficiency by concentrating computational resources on the most relevant node interactions. Additionally, parallel and distributed computing frameworks can be leveraged to accelerate training and inference. More efficient algorithms for attention computation and dynamic edge updates can also be integrated to optimize performance on large-scale graphs. Collectively, these advancements facilitate the practical application of the proposed method across diverse industrial settings with complex topologies and extensive event interactions, while maintaining computational efficiency and predictive accuracy.

Although modeling workshop production states and their dynamic interrelations through event-based representations significantly improves the predictive accuracy of future system states, several limitations affect the model's extensibility and generalization. A primary limitation is the coarse granularity of event classification, which may fail to capture subtle but important variations within production processes. For example, downtime events are often treated as a single broad category, despite differing underlying causes that can result in varying durations and impacts. This coarse attribution restricts the model's ability to differentiate among diverse factors within similar event types, reducing the precision of causal inference and long-term predictions. Another significant challenge stems from the sparsity of event occurrences. Infrequent but critical events may not be adequately recognized or learned by the model due to limited data availability. This scarcity constrains the model's sensitivity to rare but impactful production states, affecting its robustness in real-world scenarios. Moreover, the framework currently does not fully address the heterogeneity of event types, such as variable product processing times, machine-specific downtime reasons, or operator-related disruptions, all of which can substantially influence system dynamics. Despite enhancements in temporal modeling through Transformer modules and parallel long-short term dependency mechanisms, the fixed nature of these components may limit adaptability in highly variable or evolving production environments. Furthermore, while the model employs graph attention networks (GAT) to learn causal weights adaptively, these weights can alternatively be established via more precise, domain-informed modeling approaches. Such strategies can accelerate convergence and improve fitting efficiency, thus supporting scalability. To address these limitations and enhance extensibility, future work will emphasize refining event categorization by incorporating more detailed and distinct event types, including differentiated product processing phases and specific

downtime classifications. This increased granularity is expected to better capture event-specific temporal patterns and causal influences, thereby improving generalization across varied manufacturing contexts and enhancing resilience against data quality issues and event sparsity. Additionally, exploring adaptive architectures capable of dynamically adjusting temporal dependency modeling in response to operational changes will further strengthen prediction accuracy and practical applicability. The proposed framework can be naturally extended to incorporate finer-grained event labels. Specifically, the causal weight matrix A can evolve from a coarse-grained representation to a more structured form through several methodological enhancements. First, node refinement allows each downtime event to be decomposed into subtype-specific nodes, such as failure-induced or maintenance-induced categories, leading to a more expressive event graph. Second, type-aware causal modeling ensures that entries in A are conditioned on both temporal co-occurrence and event types to capture heterogeneous causal strengths. Additionally, regularization techniques like parameter sharing can be introduced to mitigate data sparsity for rare events, while a dynamic updating mechanism allows A to adapt to changing production conditions as a time-varying matrix. Automated or semi-supervised methods for event type adaptation and scalable graph construction will also be investigated to facilitate deployment across diverse industrial scenarios.

5. Conclusion

In this work, we propose the Spatial-Temporal Neural Network with Event Causal Graphs (ECGBSTNN) for predicting bottlenecks in complex manufacturing systems. Our method integrates Graph Neural Networks to capture dynamic causal relationships between events with Transformer-based techniques employing a causal mask to model long-term temporal dependencies. By representing events as nodes in a dynamic, weighted, directed graph where edges denote uncertain causal interactions, ECGBSTNN effectively leverages both causal and temporal event information. Experimental results demonstrate that this approach delivers accurate and interpretable bottleneck predictions, with causal graphs clarifying inter-event dependencies to enhance both prediction performance and explainability.

From an academic perspective, this study advances spatial-temporal modeling by embedding causal inference within neural network architectures and addressing the critical challenge of interpretability in deep learning for complex systems. The integration of dynamic event causal graphs with GNN and Transformer frameworks not only improves prediction accuracy but also enhances model transparency, paving the way for future research on explainable, causally grounded machine learning models applicable across diverse dynamic processes. From an industrial standpoint, ECGBSTNN addresses a fundamental challenge in modern manufacturing by accurately predicting bottlenecks within highly dynamic and interconnected production systems. By providing interpretable insights into the causal mechanisms underlying system inefficiencies, our approach empowers practitioners to optimize workflows, reduce costly downtime, and allocate resources more effectively. This contributes to smarter, data-driven manufacturing practices that enhance productivity and competitiveness in rapidly evolving industrial environments.

Based on these advances, we further present a streamlined framework for industrial bottleneck prediction leveraging event evolution analysis. This framework offers practical guidance for optimizing operations and maintenance, enabling operators to make informed, data-driven decisions by proactively identifying bottlenecks, improving operational efficiency, minimizing downtime, and supporting robust decision-making in complex production systems.

CRedit authorship contribution statement

Weihong Cen: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal

analysis, Data curation, Conceptualization. **Lie Yang:** Writing – review & editing, Methodology, Formal analysis. **Chupeng Su:** Writing – review & editing, Software, Methodology. **Gang Chen:** Writing – review & editing, Validation, Supervision, Resources, Funding acquisition, Formal analysis, Conceptualization. **Longhan Xie:** Supervision, Resources, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by Guangdong S&T Program (Grant No. 2026B0101110003), Natural Science Foundation of Guangdong Province (Grant No. 2024A1515011190), Research Foundation of Guangdong Province (Grant No. 2019A050505001), the Key Laboratory of Guangdong Medical Products Administration (Grant No. 2023ZDZ10), the National Key R&D Program of China (Grant No. 2021YFB3301400), the National Science Foundation of China (Grant No. 52305105), and the Basic and Applied Basic Research Foundation of Guangdong Province (Grant No. 2023A1515010812 and No. 2026A1515011789).

Data availability

The data that has been used is confidential.

References

- [1] J. Ma, Y. Shang, Z. Liang, Digital transformation, artificial intelligence and enterprise innovation performance, *Finance Res. Lett.* 78 (2025) 107190.
- [2] J. Xue, G. Li, D. Ivanov, Digital transformation in the blockchain era: balancing efficiency and resilience in operations management, *Int. J. Prod. Econ.* (2025) 109525.
- [3] J. Zhang, J. Liu, C. Zhuang, H. Guo, H. Ma, A data-driven smart management and control framework for a digital twin shop floor with multi-variety multi-batch production, *Int. J. Adv. Manuf. Technol.* 131 (11) (2024) 5553–5569.
- [4] E. Mahmoodi, M. Fathi, A.H.C. Ng, A. Dolgui, Predictive model-based multi-objective optimization with life-long meta-learning for designing unreliable production systems, *Comput. Oper. Res.* 178 (2025).
- [5] A. Skoogh, M. Thüner, M. Subramanian, A. Matta, C. Roser, Throughput bottleneck detection in manufacturing: a systematic review of the literature on methods and operationalization modes, *Prod. Manuf. Res.* 11 (1) (2023) 2283031.
- [6] M. Kumbhar, A.H.C. Ng, S. Bandaru, A digital twin based framework for detection, diagnosis, and improvement of throughput bottlenecks, *J. Manuf. Syst.* 66 (2023) 92–106.
- [7] J. Tang, Y. Liu, K.-Y. Lin, L. Li, Process bottlenecks identification and its root cause analysis using fusion-based clustering and knowledge graph, *Adv. Eng. Inform.* 55 (2023) 101862.
- [8] E. Mahmoodi, M. Fathi, M. Ghobakhloo, The impact of Industry 4.0 on bottleneck analysis in production and manufacturing: current trends and future perspectives, *Comput. Ind. Eng.* 174 (2022) 108801.
- [9] L. Ragazzini, E. Negri, L. Fumagalli, M. Macchi, Digital twin-based bottleneck prediction for improved production control, *Comput. Ind. Eng.* 192 (2024) 110231.
- [10] M. Frantzen, S. Bandaru, A.H.C. Ng, Digital-twin-based decision support of dynamic maintenance task prioritization using simulation-based optimization and genetic programming, *Decis. Anal. J.* 3 (2022) 100039.
- [11] E. Mahmoodi, M. Fathi, M. Tavana, M. Ghobakhloo, A.H.C. Ng, Data-driven simulation-based decision support system for resource allocation in Industry 4.0 and smart manufacturing, *J. Manuf. Syst.* 72 (2024) 287–307.
- [12] M. Subramanian, A. Skoogh, A.S. Muhammad, J. Bokrantz, B. Johansson, C. Roser, A data-driven approach to diagnosing throughput bottlenecks from a maintenance perspective, *Comput. Ind. Eng.* 150 (2020) 106851.
- [13] W. Urban, P. Rogowska, Methodology for bottleneck identification in a production system when implementing TOC, *Eng. Manag. Prod. Serv.* 12 (2) (2020).
- [14] X. Lai, H. Shui, D. Ding, J. Ni, Data-driven dynamic bottleneck detection in complex manufacturing systems, *J. Manuf. Syst.* 60 (2021) 662–675.
- [15] X. Lai, T. Qiu, H. Shui, D. Ding, J. Ni, Predicting future production system bottlenecks with a graph neural network approach, *J. Manuf. Syst.* 67 (2023) 201–212.
- [16] L. Li, Q. Chang, G. Xiao, S. Ambani, Throughput bottleneck prediction of manufacturing systems using time series analysis, *J. Manuf. Sci. Eng.* 133 (2) (Apr 2011) 021015.
- [17] C.A.O. Zhengcai, D. Jijie, L.I.U. Min, W.A.N.G. Yongji, Bottleneck prediction method based on improved adaptive network-based fuzzy inference system (ANFIS) in semiconductor manufacturing system, *Chin. J. Chem. Eng.* 20 (6) (2012) 1081–1088.

- [18] D. Liu, Y. Guo, S. Huang, S. Wang, T. Wu, Dynamic production bottleneck prediction using a data-driven method in discrete manufacturing system, *Adv. Eng. Inform.* 58 (2023) 102162.
- [19] W. Fang, Y. Guo, W. Liao, S. Huang, N. Yang, J. Liu, A parallel gated recurrent units (P-GRUs) network for the shifting lateness bottleneck prediction in make-to-order production system, *Comput. Ind. Eng.* 140 (2020) 106246.
- [20] J. Yin, R. Ma, S. Ge, Predicting task bottlenecks in digital manufacturing enterprises based on spatio-temporal graph convolutional networks, *Front. Eng. Manag.* (2025) 1–18.
- [21] T. Wu, J. Li, J. Bao, Z. Jin, Knowledge fusion and spatio-temporal modeling for dynamic bottleneck prediction in sustainable production processes, *Int. J. Comput. Integr. Manuf.* (2025) 1–17.
- [22] L. Ma, T. Qu, M. Thürer, Z. Wang, M. Yuan, L. Liu, An integrated spatial-temporal neural network for proactive throughput bottleneck prediction in high-variety shops with complex job routings, *Int. J. Prod. Res.* 61 (13) (2023) 4437–4449.
- [23] K. Nadim, A. Ragab, M.-S. Ouali, Data-driven dynamic causality analysis of industrial systems using interpretable machine learning and process mining, *J. Intell. Manuf.* 34 (1) (2023) 57–83.
- [24] Q. Li, P. Jiang, J. Wang, M. Yang, Y. Yang, A kind of intelligent dynamic industrial event knowledge graph and its application in process stability evaluation, *J. Intell. Manuf.* 36 (3) (2025) 1801–1818.
- [25] C. Wang, H. Wang, X. Zhang, Q. Liu, M. Liu, G. Xu, A transformer-based industrial time series prediction model with multivariate dynamic embedding, *IEEE Trans. Ind. Inform.* 21 (2) (2025) 1813–1822.
- [26] T. Deleu, M. Nishikawa-Toomey, J. Subramanian, N. Malkin, L. Charlin, Y. Bengio, Joint Bayesian inference of graphical structure and parameters with a single generative flow network, *Adv. Neural Inf. Process. Syst.* 36 (2023) 31204–31231.
- [27] L. Li, Q. Chang, J. Ni, Data driven bottleneck detection of manufacturing systems, *Int. J. Prod. Res.* 47 (18) (2009) 5019–5036.
- [28] M. Subramaniyan, A. Skoogh, J. Bokrantz, M.A. Sheikh, M. Thürer, Q. Chang, Artificial intelligence for throughput bottleneck analysis—state-of-the-art and future directions, *J. Manuf. Syst.* 60 (2021) 734–751.
- [29] C. Yu, A. Matta, A statistical framework of data-driven bottleneck identification in manufacturing systems, *Int. J. Prod. Res.* 54 (21) (2016) 6317–6332.
- [30] L. Li, Q. Chang, G. Xiao, S. Ambani, Throughput bottleneck prediction of manufacturing systems using time series analysis, *J. Manuf. Sci. Eng.* 133 (2) (2011).
- [31] M. Subramaniyan, A. Skoogh, H. Salomonsson, P. Bangalore, J. Bokrantz, A data-driven algorithm to predict throughput bottlenecks in a production system based on active periods of the machines, *Comput. Ind. Eng.* 125 (2018) 533–544.
- [32] X. Lai, H. Shui, J. Ni, A two-layer long short-term memory network for bottleneck prediction in multi-job manufacturing systems, in: *International Manufacturing Science and Engineering Conference*, 51371, 2018, page V003T02A014.
- [33] W. Cen, C. Su, K. Cen, L. Yang, G. Chen, L. Xie, A dual spatial temporal neural network for bottleneck prediction in manufacturing systems, *Eng. Appl. Artif. Intell.* 159 (2025) 111586.
- [34] B. Zhou, X. Shen, Y. Lu, X. Li, B. Hua, T. Liu, J. Bao, Semantic-aware event link reasoning over industrial knowledge graph embedding time series data, *Int. J. Prod. Res.* 61 (12) (2023) 4117–4134.
- [35] Y. Wang, L. Zheng, Y. Wang, Event-driven tool condition monitoring methodology considering tool life prediction based on industrial internet, *J. Manuf. Syst.* 58 (2021) 205–222.
- [36] B. Zhou, B. Hua, X. Gu, Y. Lu, T. Peng, Y. Zheng, X. Shen, J. Bao, An end-to-end tabular information-oriented causality event evolutionary knowledge graph for manufacturing documents, *Adv. Eng. Inform.* 50 (2021) 101441.
- [37] H. Wang, Z. Wu, H. Jiang, Y. Huang, J. Wang, S. Kopru, T. Xie, Groot: an event-graph-based approach for root cause analysis in industrial settings, in: *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2021, pp. 419–429.
- [38] Z. Yao, C. Pei, W. Chen, H. Wang, L. Su, H. Jiang, Z. Xie, X. Nie, D. Pei, Chain-of-event: interpretable root cause analysis for microservices through automatically learning weighted event causal graph, in: *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 50–61.
- [39] Y. Xu, L. Wei, L. Feng, A spatial-temporal trend-event decoupling dual-channel framework for traffic flow prediction, *Expert Syst. Appl.* (2025) 128107.
- [40] X. Liu, Q. Shao, D. Chen, Long-term prediction on graph data with causal network construction, *IEEE Trans. Artif. Intell.* 5 (7) (2024) 3445–3455.
- [41] S. Yang, Q. Wu, Z. Li, K. Wang, PSTCGCN: principal spatio-temporal causal graph convolutional network for traffic flow prediction, *Neural Comput. Appl.* (2024) 1–14.
- [42] J. Li, X. Xu, X. Ding, J. Liu, B. Ran, Bayesian spatio-temporal graph convolutional network for railway train delay prediction, *IEEE Trans. Intell. Transp. Syst.* 25 (7) (2024) 8913–8208.
- [43] C. Wang, K. Zuo, S. Zhang, C. Liu, H. Peng, W. Li, Z. Shen, P. Hu, R. Wang, B. Jie, TADGCN: a time-aware dynamic graph convolution network for long-term traffic flow prediction, *Expert Syst. Appl.* 258 (2024) 125134.
- [44] Y. Xie, J. Wu, Y. Zhou, GTHP: a novel graph transformer hawkes process for spatiotemporal event prediction, *Knowl. Inf. Syst.* 66 (7) (2024) 4043–4062.
- [45] Y. Xie, J. Wu, HGTHP: a novel hyperbolic geometric transformer hawkes process for event prediction, *Appl. Intell.* 54 (1) (2024) 357–374.
- [46] S.-Y. Chiang, C.-T. Kuo, S.M. Meerkov, DT-bottlenecks in serial production lines: theory and application, *IEEE Trans. Robot. Autom.* 16 (5) (2000) 567–580.
- [47] M. Subramaniyan, A. Skoogh, H. Salomonsson, P. Bangalore, M. Gopalakrishnan, A. Sheikh Muhammad, Data-driven algorithm for throughput bottleneck analysis of production systems, *Prod. Manuf. Res.* 6 (1) (2018) 225–246.
- [48] Z. Cao, J. Deng, M. Liu, Y. Wang, Bottleneck prediction method based on improved adaptive network-based fuzzy inference system (ANFIS) in semiconductor manufacturing system, *Chin. J. Chem. Eng.* 20 (6) (2012) 1081–1088.